

=====

✂ LWC & APEX EXAM SYNTAX BLUEPRINT CHEAT SHEET ✂

=====

1. HTML TEMPLATE MARKUP MARKUP MARKS SYNTAX

[Standard List Loop (for:each)]

```
<template for:each={List} for:item="it">
  <!-- Every immediate child tag inside a loop MUST have a unique key -->
  <div key={it.Id} class="slds-box slds-m-vertical_x-small">
    <p>{it.Name}</p>
  </div>
</template>
```

[Conditional UI Rendering (lwc:if / lwc:else)]

```
<template lwc:if={hasRecords}>
  <p>Pipeline records found!</p>
</template>
<template lwc:else>
  <p>No records pass forecasting audit constraints.</p>
</template>
```

[List Row Interactive Button Trigger Configuration]

```
<lightning-button
  label="button"
  variant="variant_type"
  data-id={it.Id}
  onclick={functionName}>
</lightning-button>
```

[Card view]

```
<lightning-card title="title">
  <div class="slds-p-around_medium">
    <p>Name: <strong>{name}</strong></p>
    <p>Stage: <strong>{stage}</strong></p>
    <p>Notes: <strong>{notes}</strong></p>
    <p>Search Query: <strong>{searchQuery}</strong></p>
  </div>
</lightning-card>
```

[Text Area]

```
<lightning-textarea
  label="Notes"
  value={notes}
  onClick={function}
  class="slds-m-top_small">
</lightning-textarea>
```

[Combo Box]

```
<lightning-combobox
  label="Label"
  options={stageOptions}
  value={stage}
  required
  onChange={handleStageChange}
```

```
        class="slds-m-top_small">
</lightning-combobox>
```

2. JAVASCRIPT JS DECORATORS & CORE CORE INTERACTION SYNTAX

[Base LWC Library Framework Imports & Class Signature]

```
import { LightningElement, api, wire, track } from 'lwc';
```

```
export default class OppExamDashboard extends LightningElement {
  @api recordId; // Captures current record ID context page natively
}
```

[Pure JS Data Getters (Calculations & Formatting Transforms)]

// Getters MUST feature a 'return' keyword and accept zero arguments

```
get uppercaseName() {
  return this.someVariable ? this.someVariable.toUpperCase() : "";
}
```

[Accessing Custom Data-Attributes (data-id) inside Event Handlers]

```
handleProcessRevenue(event) {
  // Captures the unique database row pointer string from the HTML element
  const selectedOppld = event.target.dataset.id;
}
```

3. PLATFORM INTEGRATION SERVICES & APEX INTEGRATION CODE BLUEPRINTS

[Secure Module Framework Path Mapping Imports]

```
import getClosingOppsList from '@salesforce/apex/OppExamController.getClosingOppsList';
import setExpectedRevenueLogic from
  '@salesforce/apex/OppExamController.setExpectedRevenueLogic';
```

```
import { refreshApex } from '@salesforce/apex';
import { ShowToastEvent } from 'lightning/platformShowToastEvent';
import { NavigationMixin } from 'lightning/navigation';
```

[Hooking Apex Controllers via Reactive Wires (Read Paths)]

```
// Wires provision whole tracking wrapper objects inside 'wiredOppsResult'
@wire(getClosingOppsList)
wiredOppsResult;
```

```
get opportunityList() {
  return this.wiredOppsResult?.data || [];
}
```

[Imperative Server Invocations using Asynchronous async/await (Action Paths)]

// 1. Mark the outer event handler function explicitly as async

```
async handleProcessRevenue(event) {
  const selectedOppld = event.target.dataset.id;
```

```
  // 2. Wrap all imperative execution promises inside an error catch block
  try {
```

```
    // 3. Pause runtime execution using await to block until database returns
```

```

        await setExpectedRevenueLogic({ oppId: selectedOppId });

        // 4. Force a clean cache reload by passing the full provisioned wire wrapper
        await refreshApex(this.wiredOppsResult);

    } catch (error) {
        console.error('Database process failed:', error.body.message);
    }
}

```

[Success Confirmation Popups (Toast Events)]

ShowToastEvent is the event name of Toast which has title, message and variant

[Redirection Routing Framework Layout (NavigationMixin Body)]

// 1. Class signature wrapper MUST extend NavigationMixin (e.g., NavigationMixin(LightningElement))

```

this[NavigationMixin.Navigate]({
    type: "",
    attributes: {
        objectApiName: "",
        actionName: ""
    }
});

```

[]

4. APEX SERVER-SIDE CONTROLLER CLASS SYNTAX

// Always explicitly include "with sharing" for record-level visibility gates
 public with sharing class OppExamController {

```

        @annotation(handle caching here)
        public static List<object> methodName() {
            return statement;
        }
    }
}

```

5. METADATA DEPLOYMENT CONFIGURATION SETTINGS (.js-meta.xml)

lightning__RecordPage for object's record page
 lightning__HomePage for object's home page

=====